Advanced Database Systems: Final Portfolio Summary

MIT 8103 | 2026/2027 First Semester | Individual Practical Database Portfolio

Osisiogu Nnabugwu | Student ID: 301930508

https://github.com/celetx/advanced-database-portfolio (public)

Selected organisation and scope

MediCare Health Clinic is a simulated healthcare organisation used consistently across all five activities. Receptionists register patients and arrange appointments; healthcare providers record visits and services; finance officers manage payments; administrators maintain the system and review reports. Major data includes patients, providers, appointments, clinical visits, medical services, delivered-service quantities and payments. Key operations include booking appointments, retrieving patient history, recording charges and payments, transferring internal clinic funds and analysing service usage. All records are fictional. Portfolio 5 extends the scenario to fictional Ikeja, Lekki and Abuja branches.

The portfolio progresses from relational design and query optimisation to transaction control, MongoDB modelling and PostgreSQL replication. SQL scripts, configuration, READMEs, execution outputs and screenshots are retained in the public repository with progressive commits. The final documentation covers all five activities, as required by Sections 4 and 6 of the lecturer’s assessment brief.

Portfolio 1 — Database design and modelling

A PostgreSQL schema represents seven related tables: Patients, Providers, Appointments, ClinicVisits, MedicalServices, VisitServices and Payments. Primary and foreign keys express relationships, while UNIQUE, NOT NULL and CHECK constraints control names, required values, quantities, payment methods and monetary amounts. An ER diagram and normalisation explanation cover 1NF–3NF. Five recorded negative tests rejected duplicate keys, invalid references, missing patient names, negative fees and duplicate catalogue services. This provides the integrity foundation used in the later exercises.

Portfolio 2 — Query processing and optimisation

SQL joins and aggregates report appointments, provider activity, completed-payment revenue and service usage. A synthetic dataset added 2,000 patients and 100,000 appointments. EXPLAIN (ANALYZE, BUFFERS) was used before and after B-tree indexing. The recorded appointment-date query fell from 4.747 to 0.645 ms (86.41% reduction); patient lookup fell from 6.792 to 0.086 ms (98.73%). Index-assisted plans examined fewer candidate rows. These are the existing environment-specific observations, not a universal speed guarantee or a new benchmark performed for this summary.

Portfolio 1 evidence

Portfolio 2 query plans and results

Portfolio 3 — Transactions and concurrency

The PostgreSQL account exercise begins with NGN 100,000 and NGN 50,000. A committed NGN 10,000 transfer produces NGN 90,000 and NGN 60,000 while preserving the NGN 150,000 total; a rolled-back transfer leaves the committed balances unchanged. Separate sessions demonstrate that READ COMMITTED can observe a later committed update, while REPEATABLE READ retains its transaction snapshot. A held row lock causes a competing update to reach its configured five-second lock timeout. After the lock is released, a retry succeeds. These experiments support the discussion of atomicity, isolation and the need to handle failed transactions.

Transaction scripts, session instructions and screenshots

Portfolio 4 — NoSQL and advanced data models

MongoDB 8.0.16 implements patients, providers, services, appointments and visits as five collections. Visits embed bounded service and payment arrays, including historical service-price snapshots; references avoid repeating shared patient and provider identities. Extended JSON dates become BSON dates, and whole-kobo amounts avoid fractional currency arithmetic. Validators and indexes check document shape and selected local rules, while cross-collection reference integrity remains an application responsibility.

CRUD operations, four aggregations, invalid-document tests, reference audits and a hinted compound-index plan were executed. Both the recorded cloud run and the student’s Codespaces log report 25 passing assertions. Completed revenue totals NGN 22,000. The comparison with PostgreSQL discusses integrity, atomicity, schema flexibility and workload suitability without claiming that MongoDB is universally faster.

MongoDB implementation and Codespaces execution log

Portfolio 5 — Distributed and cloud database exercise

PostgreSQL 16.11 physical WAL streaming connects a primary and a hot standby, using separate processes, ports and data directories in one isolated Docker container. The exercise verifies insert/update replay, committed and rolled-back account transfers, read-only enforcement, replica interruption/catch-up, surviving reads, fenced manual promotion, rejoining from a fresh base backup, and logical backup/restore. All ten table content hashes match at the consistency checkpoints and after restore. The student’s Codespaces run reports 37 passing assertions and the final success verdict.

This is genuine replication on one host, not geographically independent infrastructure. Asynchronous commits do not guarantee an immediately current replica; controlled promotion does not establish zero data loss under arbitrary failures. Manual promotion is not automatic HA. A proposed Amazon RDS design covers private access, managed availability, reporting replicas, backups, monitoring, costs and Nigerian privacy considerations. No managed RDS instance, cloud PITR or multi-AZ failure was deployed or tested.

Replication implementation, evaluation and cloud proposal

Selected practical evidence

Figure 1 is the student-uploaded Codespaces terminal capture from Portfolio 5. It shows restore checks, measured observations and ALL PORTFOLIO 5 TESTS PASSED. Full logs and additional screenshots remain in the repository; this image is not a synthetic terminal or a cloud-deployment confirmation.

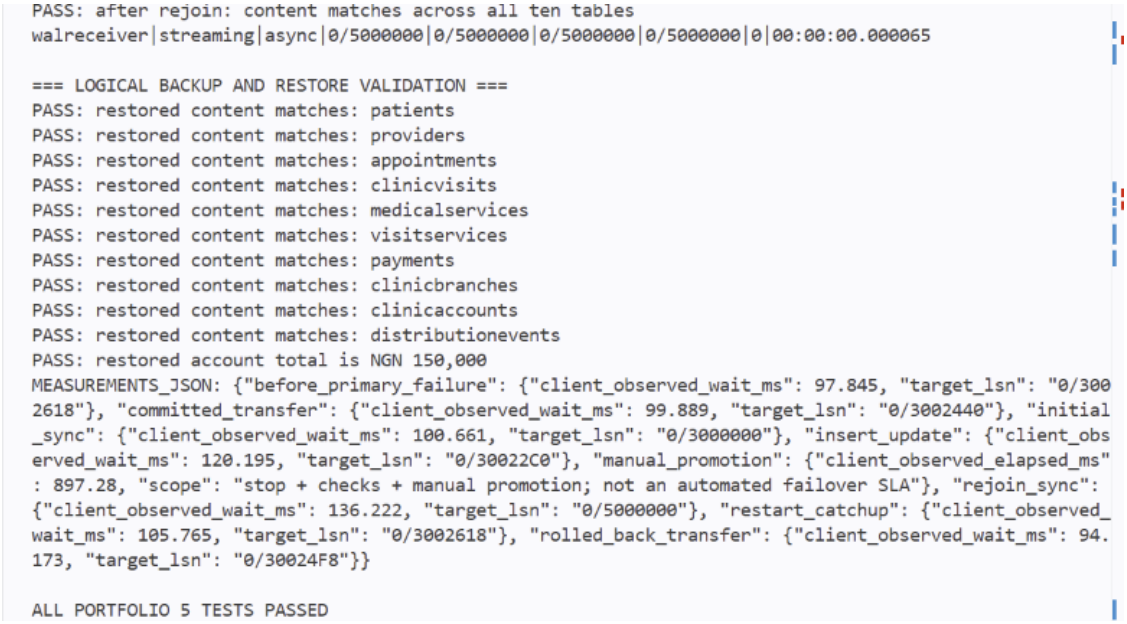


Figure 1. Genuine Portfolio 5 Codespaces terminal screenshot; source file linked below.

Full-size Codespaces screenshot

Saved Codespaces execution log (37 passing assertions)

Individual technical reflection

Using related tables in PostgreSQL was a design choice I found particularly interesting. Patients, providers, appointments and payments are stored in separate tables and linked through keys. This reduces unnecessary duplication and helps prevent invalid references, such as an appointment referring to a patient who does not exist. For me, this is an interesting and standard approach to real-life database implementation. I ran the MongoDB and PostgreSQL exercises in Codespaces, checked their success messages and committed the saved logs and a PostgreSQL terminal screenshot. I also revised the README locally. When Git prevented a branch switch because of those edits, guided stash and restore commands helped me recover and publish my preferred version. This highlighted why local edits, saved commits and remote pushes must be treated as separate steps.

References and AI-use declaration

Technical references: PostgreSQL documentation (constraints, EXPLAIN, transaction isolation and streaming standby); MongoDB documentation (data modelling, validation and aggregation); AWS RDS documentation (read replicas, backups and TLS). Nigerian privacy considerations refer to the NDPA 2023 and NDPC GAID 2025; no compliance certification is claimed. Detailed source links are maintained in the Portfolio 4 comparison and Portfolio 5 References.md.

OpenAI Codex assisted Portfolio 4–5 design, implementation, testing, explanations and evidence preparation, and helped prepare this combined summary from the existing Portfolios 1–3. The student reviewed and edited the README, reproduced Portfolios 4 and 5 in Codespaces, checked the success outputs and committed the logs and PostgreSQL screenshot. AI assistance does not replace the student's responsibility for accuracy, source acknowledgement and individual understanding. The reflection combines the student's own PostgreSQL design observation with the verification actions recorded during the work.

PostgreSQL | MongoDB | AWS RDS | Portfolio references